Data Structures and Algorithms
Lecture 6 – Trees

M.G.Noel A.S Fernando (PhD)

Professor

UCSC

Trees

- A *tree* data structure is a powerful tool for organizing data objects based on keys.
- It is equally useful for organizing multiple data objects in terms of hierarchical relationships (think of a ``family tree'', where the children are grouped under their parents in the tree).

Trees

Further, Tree is one of the most important non-linear data structures in computing. It allows us to implement faster algorithms(compared with algorithms using linear data structures).

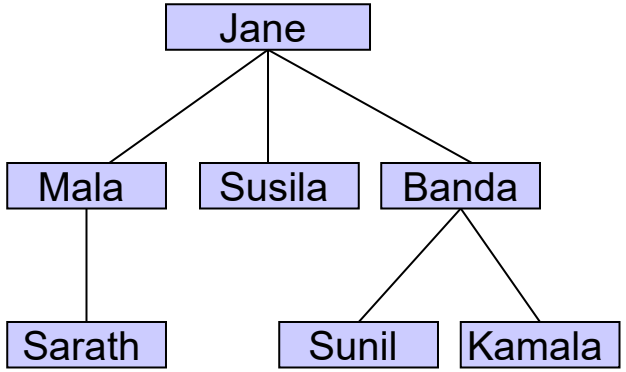
Application areas are :

- Almost all operating systems store files in trees or tree like structures.
- Compilers use a syntax tree to validate the syntax of every program you write
- Binary Search Trees(BSTs) are used to quickly check whether an element is present in a set or not(Searching Algorithms)

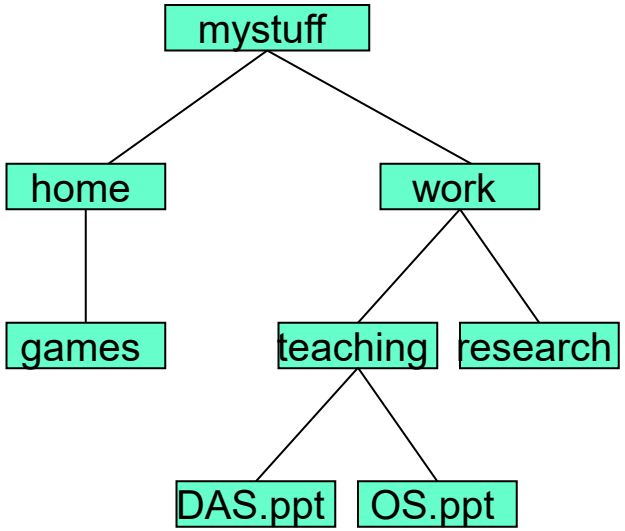
An application :File system

- There are many applications for trees.
- A popular one is the directory structure in many common operating systems, including VAX/VMX, Unix and DOS, and windows.

Trees



Jane's children and grandchildren

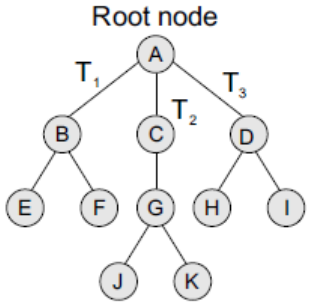


File organization in a computer

Tree Types

- Binary tree — each node has at most two children
- General tree — each node can have an arbitrary number of children.
- Binary search trees
- AVL trees

Tree Terminology and Basic Properties

- Definition : A tree is recursively defined as a set of one or more nodes where one node is designated as the root of the tree and all the remaining nodes can be partitioned into non-empty sets each of which is a sub-tree of the root.
- 

- The above figure shows a tree where node A is the root node; nodes B, C, and D are children of the root node and form sub-trees of the tree rooted at node A.

Basic Terminology

- ✓**tree** - a non-empty collection of vertices & edges

- ✓**vertex (node)** - can have a name and carry other associated information

- ✓**path** - list of distinct vertices in which successive vertices are connected by edges

Basic Terminology

- ***Root node*** The root node R is the topmost node in the tree. If R = NULL, then it means the tree is empty
- ***Sub-trees*** If the root node R is not NULL, then the trees T1, T2, and T3 are called the sub-trees of R.
- ***Leaf node*** A node that has no children is called the leaf node or the terminal node.
- ***Path*** A sequence of consecutive edges is called a *path*. For example, in the above diagram, the path from the root node A to node I is given as: A, D, and I.

Basic Terminology cont....

- ***Ancestor node*** An ancestor of a node is any predecessor node on the path from root to that node.
- The root node does not have any ancestors. In the tree given in the above diagram, nodes A, C, and G are the ancestors of node K.
- ***Descendant node*** A descendant node is any successor node on any path from the node to a leaf node.
- Leaf nodes do not have any descendants. In the tree given in above, nodes C, G, J, and K are the descendants of node A.

Basic Terminology cont....

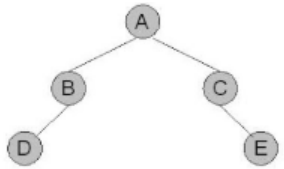
- ***Level number*** Every node in the tree is assigned a *level number* in such a way that the root node is at level 0, children of the root node are at level number 1.
- Thus, every node is at one level higher than its parent. So, all child nodes have a level number given by parent's level number + 1.

Basic Terminology cont….

- ***Degree:*** Degree of a node is equal to the number of children that a node has.
- The degree of a leaf node is zero.
- ***In-degree:*** In-degree of a node is the number of edges arriving at that node.
- ***Out-degree:*** Out-degree of a node is the number of edges leaving that node.
- e.g.

Basic Terminology cont....

- E.g.

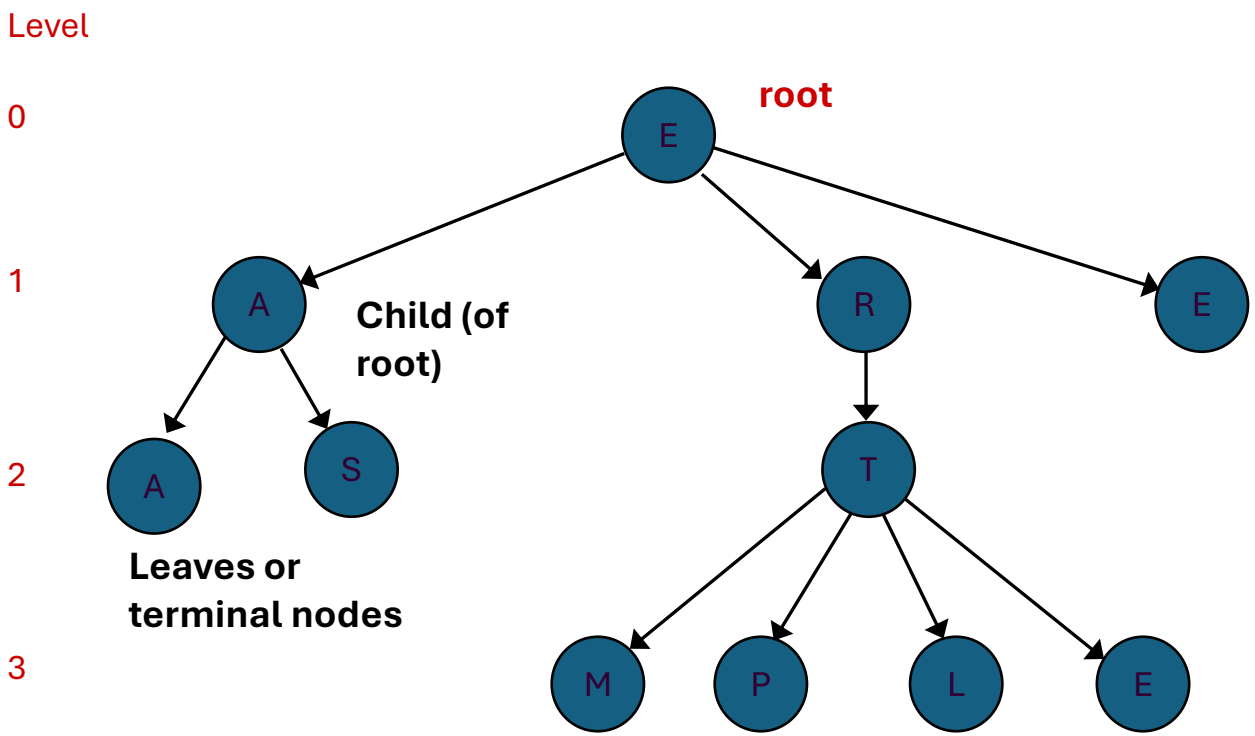


- **Indegree** of a node - The number of edges merging into a node. For example, indegree of the node B is one i.e., one edge merges.
- **Outdegree** of a node - The number of edges coming out from a node. For example, outdegree of the node A is two i.e., two edges come out of this root node.

Basic Terminology cont....

- ✓**Depth of a node** - the length of the path from the root to that node
  - **root: depth 0**
- ✓**Height of a node** - the length of the longest path from that node to a leaf
  - **any leaf: height 0**
- ✓**Height of a tree:** The length of the longest path from the root to a leaf

Trees - Example

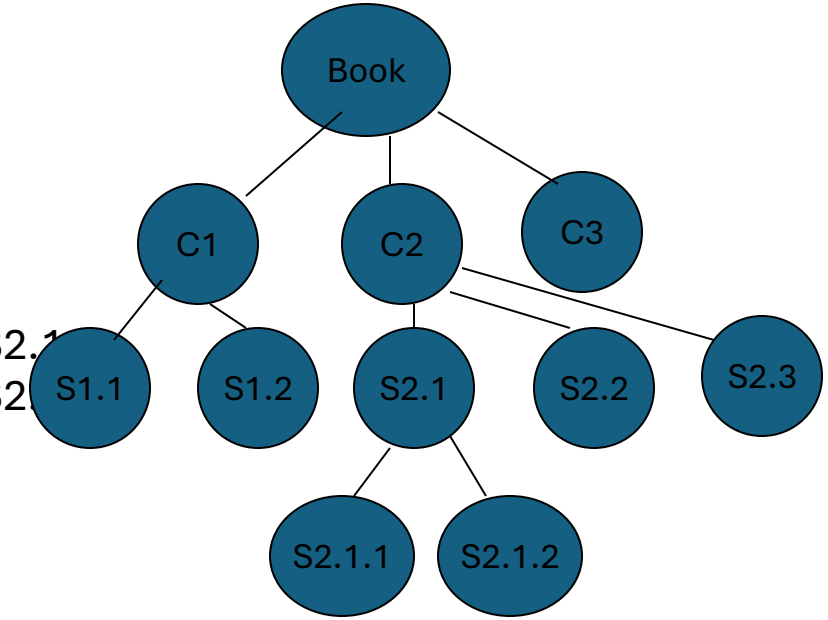


Depth of T: 2

Height of T: 1

Eg. A table of contents and its tree representation

Book
- C1
  - S1.1
  - S1.2
- C2
  - S2.1
    - S2.1.1
    - S2.1.2
  - S2.2
  - S2.3
- C3



- Degree : The number of sub tree of a node is called its degree.
- Eg. Degree of book → 3, C1→2,C3→0

- Nodes that have degree 0 is called Leaf or Terminal node.Other nodes called non-terminal nodes. Eg.Leaf nodes :C3,S1.1,S1.2 etc.

- Book is said to be the father (parent) of C1,C2,C3 and C1,C2,C3are said to be sons (children) of book.

- Children of the same parent are said to be siblings.Eg.C1,C2,C3 are siblings (Brothers)

- Length : The length of a path is one less than the number of nodes in the path.(Eg path from book to s1.1=3-1=2

.

- If there is a path from node a to node b , then **a** is an ancestor of b and b is descendent of a.
- In above example, the ancestor of S2.1are itself,C2 and book, while it descendent are itself, S2.1.1 and S2.1.2.
- An ancestor or descendent of a node, other than the node itself is called a proper ancestor or proper descendent.
- Height of a tree — the maximum depth of a leaf node. ..[In above example height=3]
- **Depth of a node** — the length of the path between the root and the node.[In above example node C1 has **Depth** 1, node S2.1.2 has **Depth** 3.etc.]

General Trees.

Trees can be defined in two ways :

- Recursive
- Non- recursive

One natural way to define a tree is recursively

General Trees.

Trees can be defined in two ways :

- Recursive
- Non- recursive

One natural way to define a tree is recursively

Definition of Tree -general tree

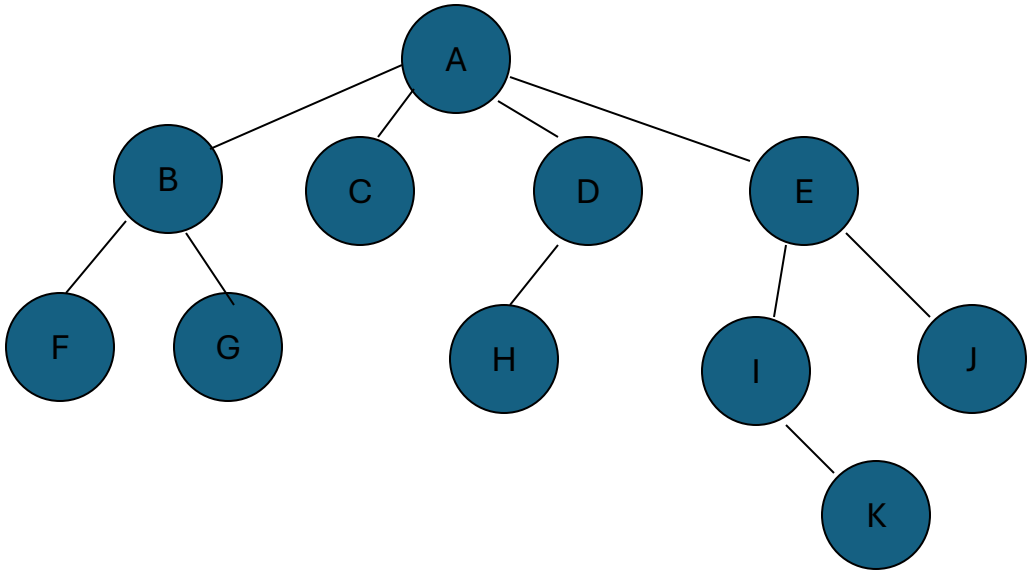
- A tree is a finite set of one or more nodes such that:
- There is a specially designated node called the root.
- The remaining nodes are partitioned into $n>=0$ disjoint sets $T_1, \ldots, T_n$, where each of these sets is a tree.
- We call $T_1, \ldots, T_n$ the subtrees of the root.

General trees-Definition

- A tree is a collection of nodes. The collection can be empty; otherwise a tree consists of a distinguish node r, called root, and zero or more non-empty (sub)trees $T_1, T_2, T_3, \ldots T_K$. · Each of whose roots are connected by a directed edge from r.

General trees-Definition

- A tree consists of <u>set of nodes</u> and <u>set of edges</u> that connected pair of nodes.



Representation of Trees

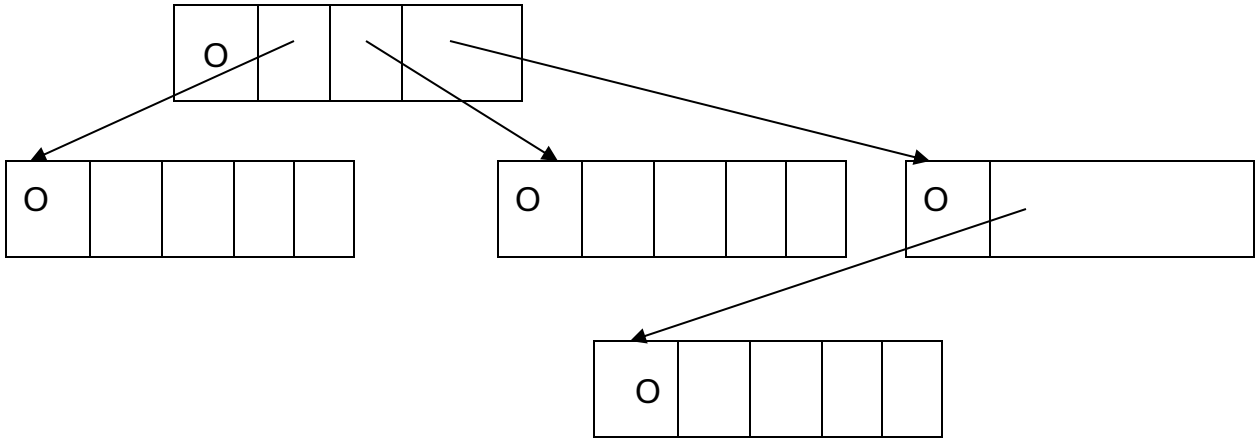
- List Representation
  - (A (B (E (K, L), F), C (G), D (H (M), I, J)))
  - The root comes first, followed by a list of sub-trees

data	link 1	link 2	...	link n

How many link fields are
needed in such a representation?

A Tree Node

- Every tree node:
  - object – useful information
  - children – pointers to its children nodes



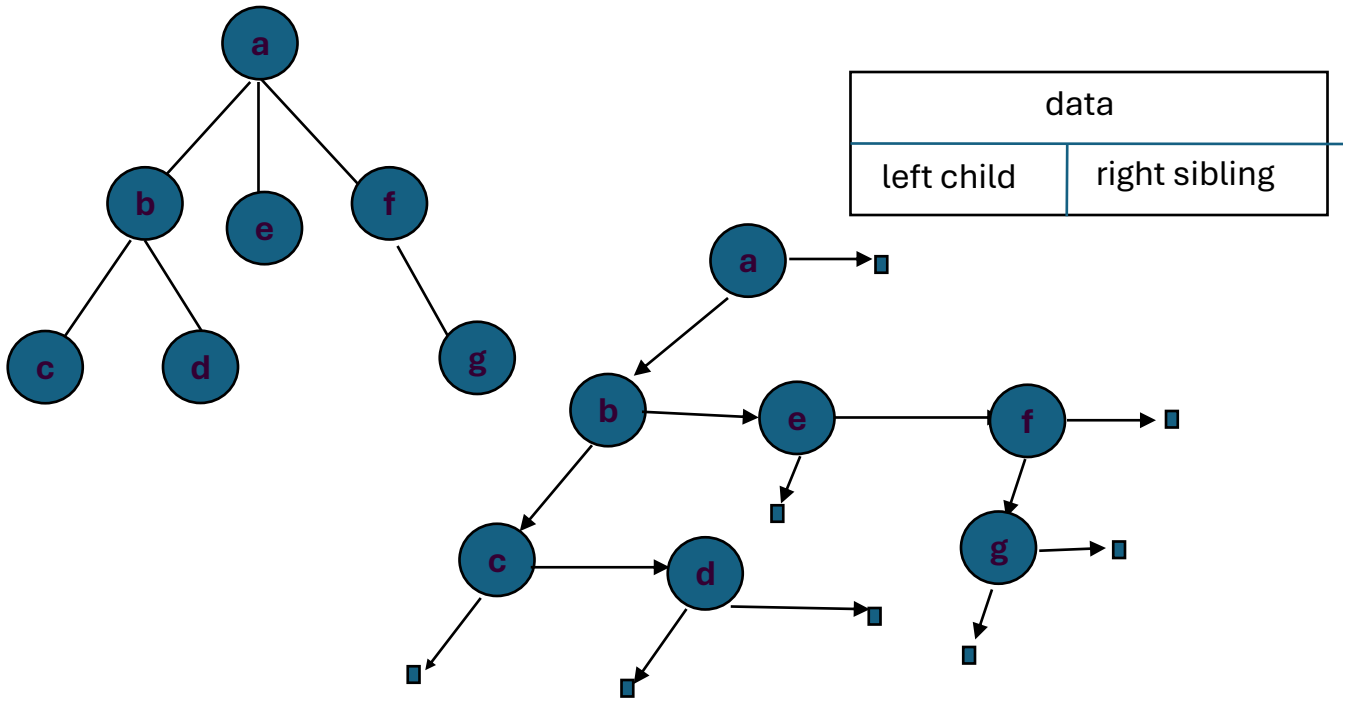
Tree - Implementation

- One way to implement a tree would be to have in each node a reference to each child of the node in addition to its data.
- However, the number of children per node can vary so greatly and is not known in advanced.
- It might be infeasible to make the children direct links in the data structure.
- These would be too much wasted space.

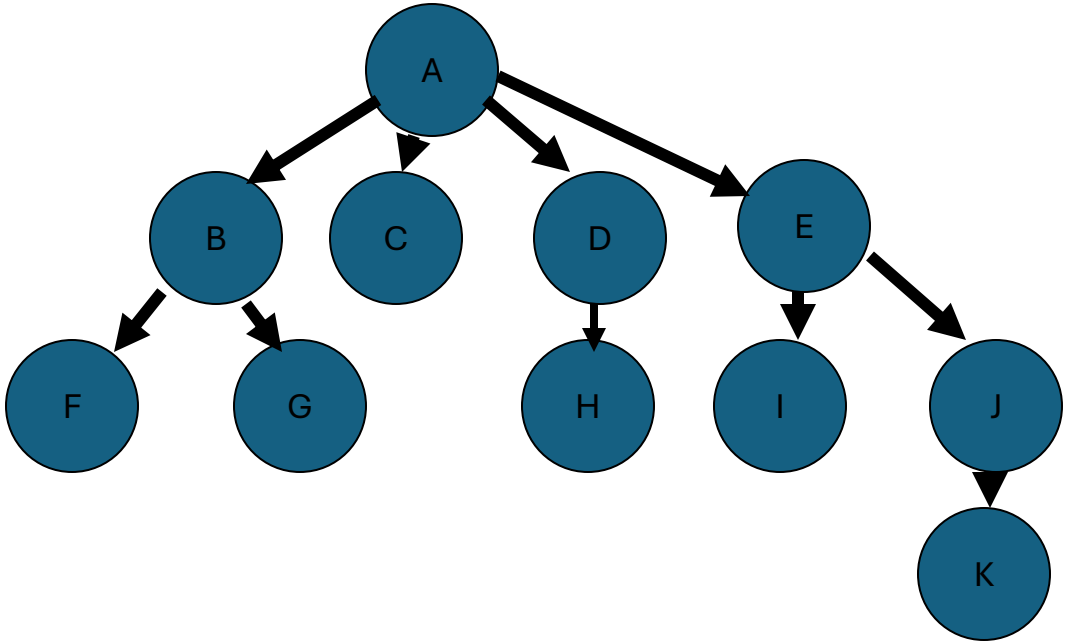
The solution is simple :

- Keep the children of each node in a linked list of tree nodes. Thus each node keeps two references : one to its leftmost child and other one for its right sibling.

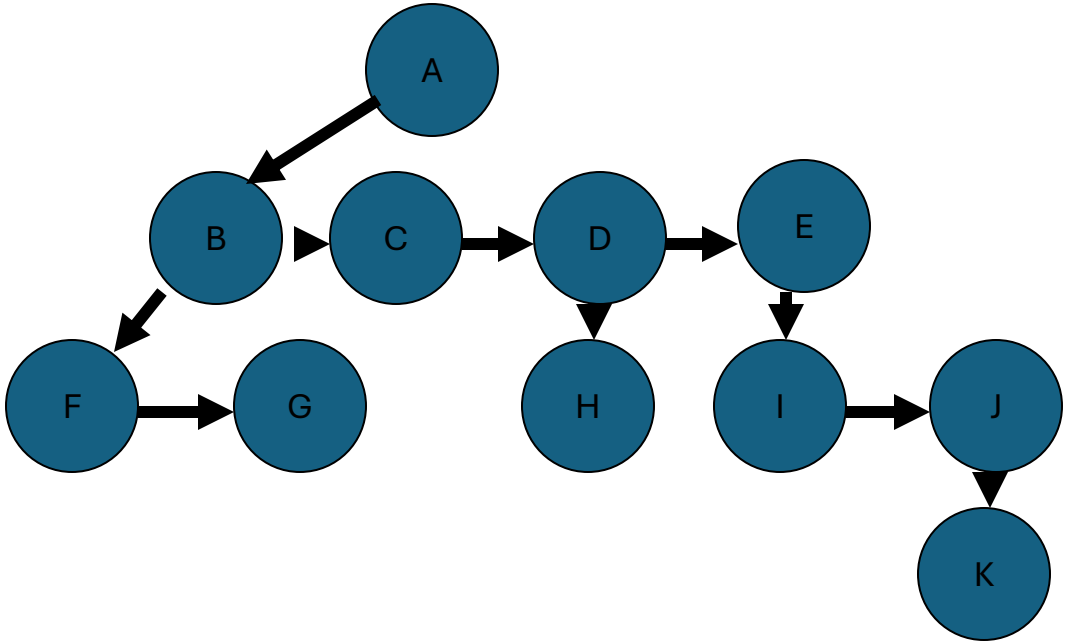
Example



Left Child - Right Sibling representation of a tree

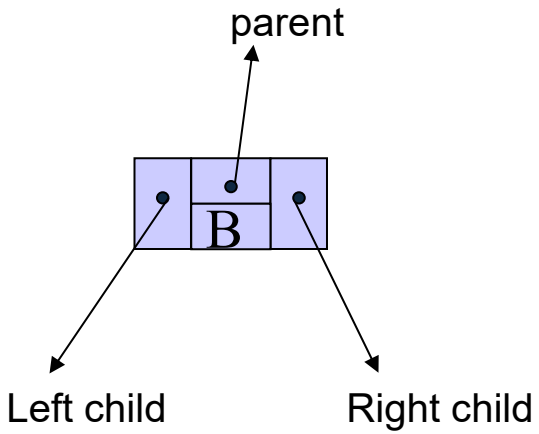


Left most Child - Right Sibling representation of a tree

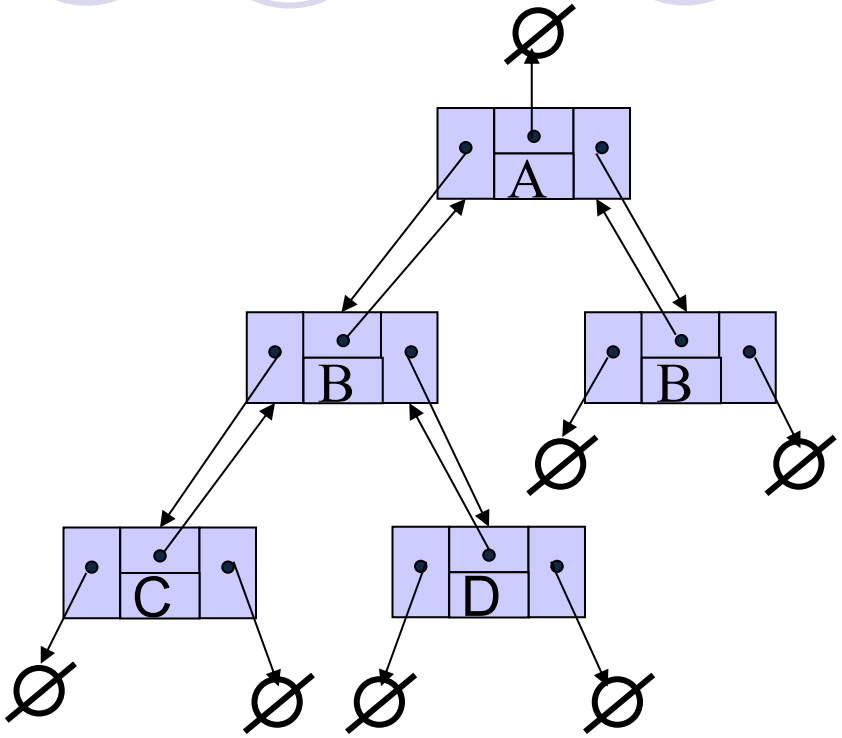


7.3.2 A Linked Structure for Binary Trees

- We represent each node of a binary tree by an object which stores
  - Element
  - References to its parent and children nodes



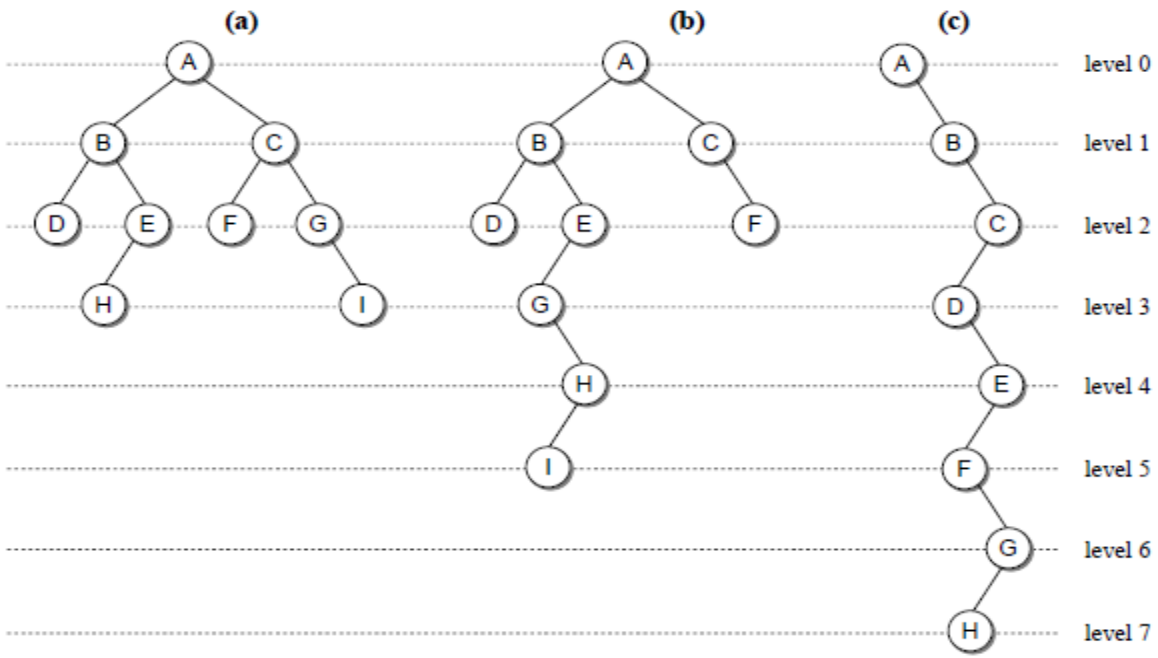
7.3.2 A Linked Structure for Binary Trees



Properties of Binary Tress

- Binary trees come in many deferent shapes and sizes. The shapes vary depending on the number of nodes and how the nodes are linked.

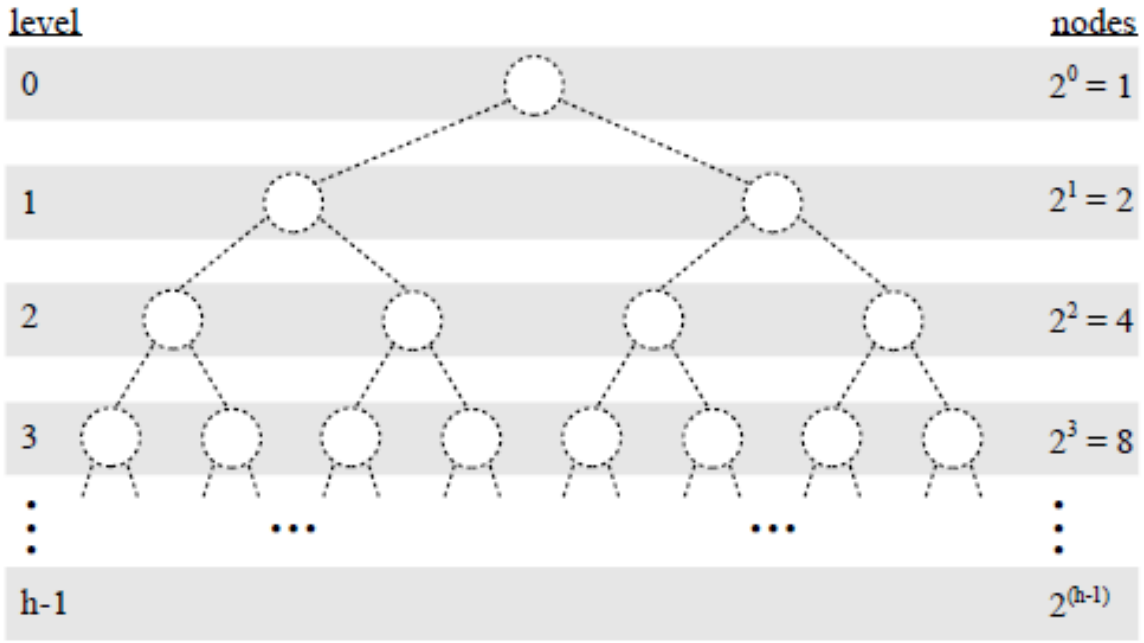
Three different arrangements of nine nodes in a binary tree.



Three different arrangements of nine nodes in a binary tree (cont..)

- Consider node G in the three trees of above Figure. In tree (a), G has a depth of 2, in tree (b) it has a depth of 3, and in (c) its depth is 6.
- The height of a binary tree is the number of levels in the tree. For example, the three binary trees in the above Figure have different heights: (a) has a height of 3, (b) has a height of 5, and (c) has a height of 7.

Possible slots for the placement of nodes in a binary tree



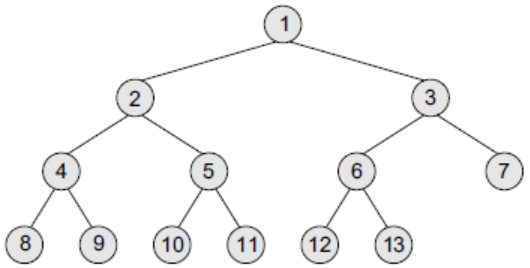
Complete Binary Trees

- A *complete binary tree* is a binary tree that satisfies following 3 properties.
- In a complete binary tree, every level, except possibly the last, is completely filled.
- all nodes appear as far left as possible.
- Each non leaf node has <u>exactly two</u> child nodes

Example

- In a complete binary tree Tn, there are exactly n nodes and level r of T can have at most 2^r nodes
- Note that in the diagram, level 0 has $2^0 = 1$ node, level 1 has $2^1 =$ 2nodes, level 2 has $2^2 = 4$ nodes,
- level 3 has 6 nodes which is less than

  the maximum of $2^3 = 8$ nodes.



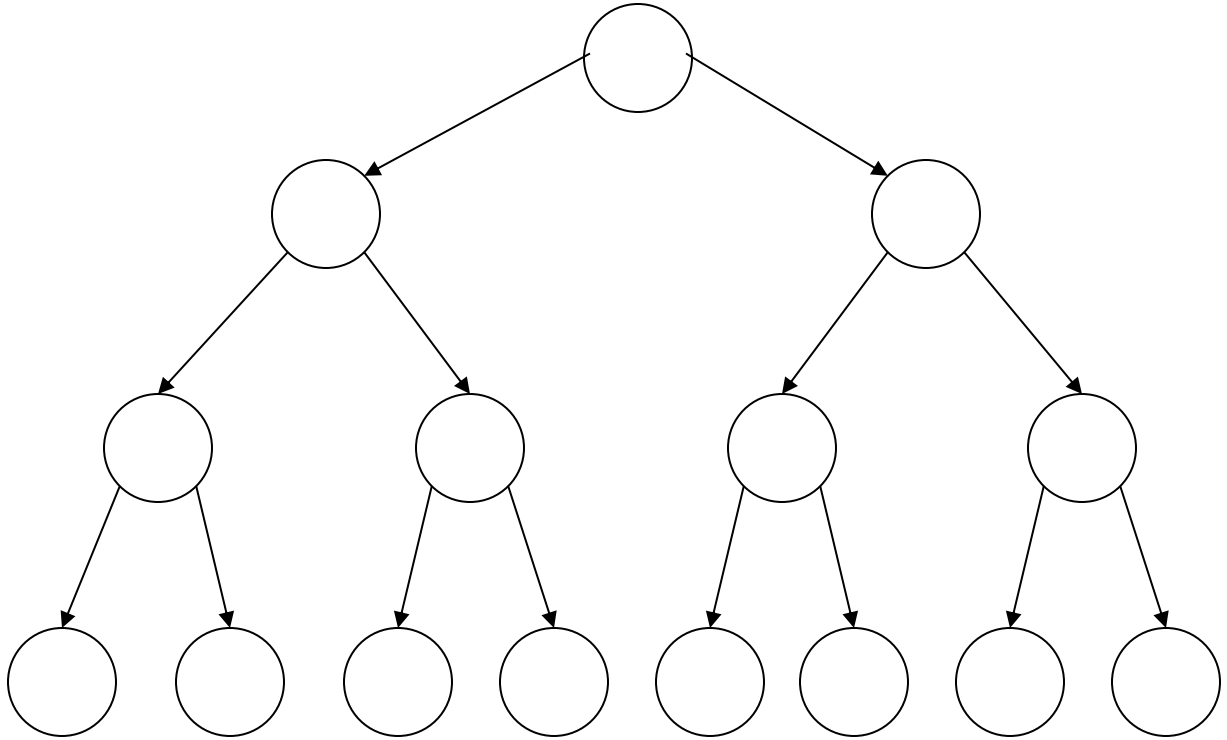
Example cont....

- In the above tree has exactly 13 nodes. They have been purposely labelled from 1 to 13,
- So that it is easy for the reader to find the parent node, the right child node, and the left child node of the given node.
- The formula can be given as—if K is a parent node, then its left child can be calculated as 2 × K and its right child can be calculated as 2 × K + 1.
- For example, the children of the node 4 are 8 (2 × 4) and 9 (2 × 4 + 1).
- Similarly, the parent of the node K can be calculated as | K/2 |.
- Given the node 4, its parent can be calculated as | 4/2 | = 2.
- The height of a tree T_n having exactly n nodes is given as:
- H_n = | log2 (n + 1) |
- if a tree (T) has 10,00,000 nodes, then its height is 21.

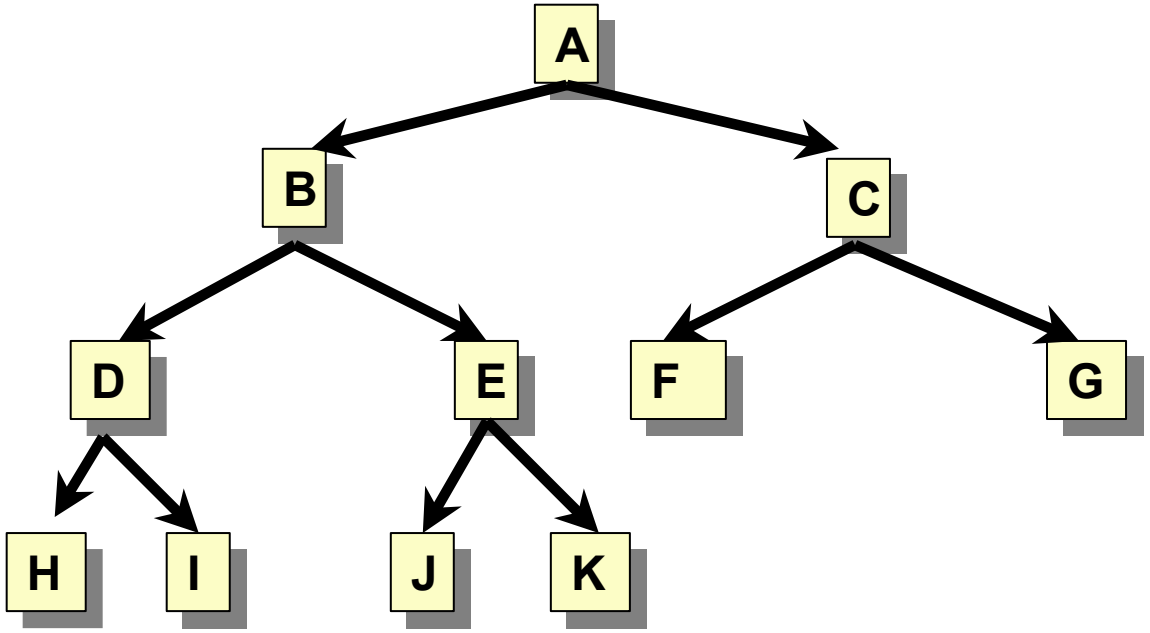
Properties of Binary Trees

- A binary tree is a **full** binary tree if and only if:
  - Each non leaf node has exactly two child nodes
  - All leaf nodes have identical path length
- It is called **full** since all possible node slots are occupied

A Full Binary Tree - Example



Complete Binary Trees - Example



Tree Terminology and Basic Properties

- Properties of Binary Trees
  - Let T be a complete binary tree with *n* nodes, and let *h* denote the height of T, then T has the following properties:
  -

	At least	At most
The number of leaf nodes	$h+1$	2^h
The number of non-leaf nodes	h	2^h-1
The total number of nodes	$2h+1$	$2^{h+1}-1$
The height of the tree h	$\log(n+1)-1$	$(n-1)/2$

Properties of binary trees.

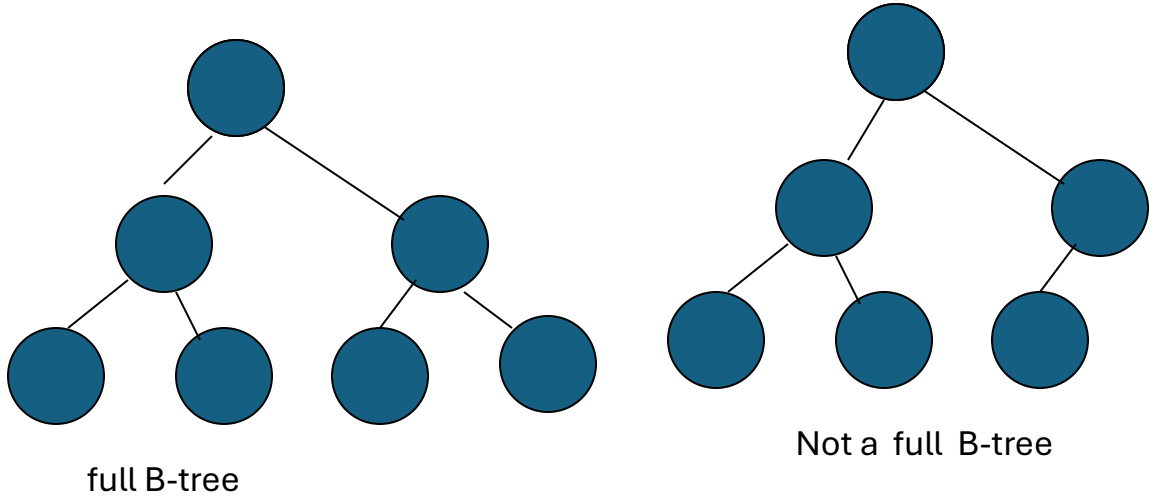
- If a binary tree contains m nodes at level L, then it contains at most 2m nodes at level L+1.
- A binary tree can contain at most 2^L nodes at L

At level 0 B-tree can contain at most 1= 2^0 nodes

At level 1 B-tree can contain at most 2= 2^1 nodes At level 2 B-tree can contain at most 4= 2^2 nodes At level L B-tree can contain at most → 2^L nodes

Full B-tree

- A full B-tree of depth d is the B-tree that contains exactly 2^L nodes at each level between 0 and d (or 2^d nodes at d)



The total number of nodes (Tn) in a full binary tree of depth d is 2^{d+1}-1

- $Tn=2^0+2^1+2^2+.....2^d.....(1)$
- $2Tn=2^1+2^2+..... 2^{d+1}....(2)$
- (2)-(1)→
- $Tn=2^{d+1}-1$

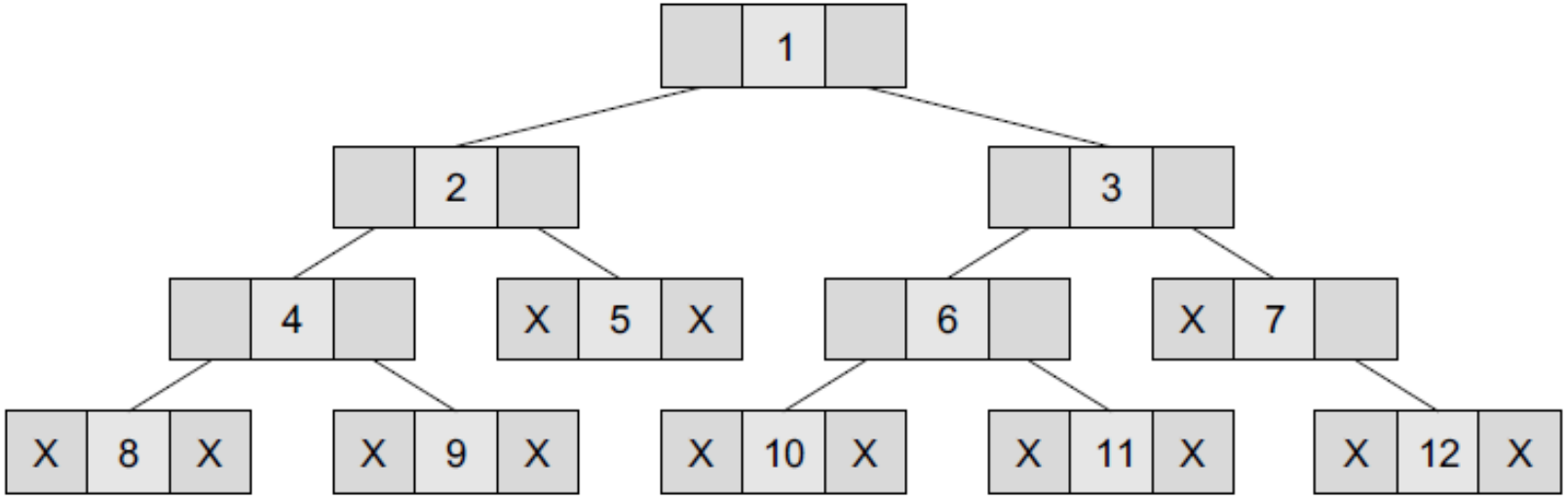
Representation of Binary Trees in the Memory

- In the computer's memory, a binary tree can be maintained either by using a linked representation or by using a sequential representation.
- ***Linked representation of binary trees***
- In the linked representation of a binary tree, every node will have three parts:
- the data element,
- a pointer to the left node, and
- a pointer to the right node.
- So in C, the binary tree is built with a node type given below

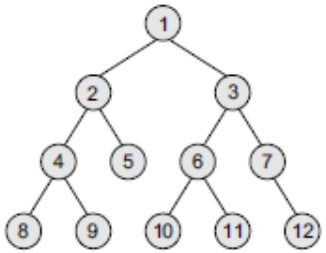
Representation of Binary Trees in the Memory cont....

```
struct node {
    struct node *left;
    int data;
    struct node *right;
};
```

Representation of Binary Trees in the Memory cont....



The above tree is represented in the main memory using a linked list as follows:

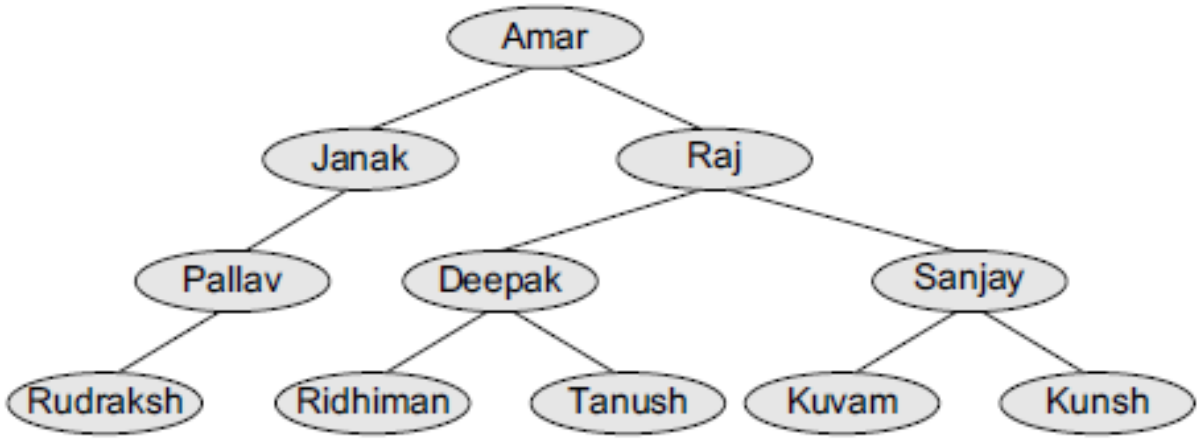


ROOT 3

15 AVAIL

	LEFT	DATA	RIGHT
1	–1	8	–1
2	–1	10	–1
3	5	1	8
4			
5	9	2	14
6			
7			
8	20	3	11
9	1	4	12
10			
11	–1	7	18
12	–1	9	–1
13			
14	–1	5	–1
15			
16	–1	11	–1
17			
18	–1	12	–1
19			
20	2	6	16

Given the memory representation of a tree that stores the names of family members, construct the corresponding tree from the given data.

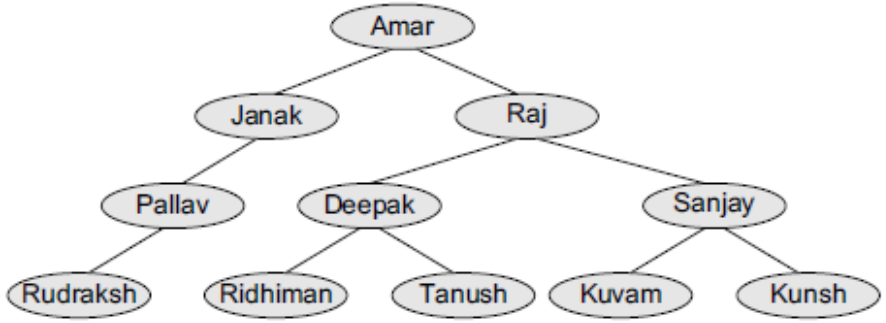


Solution :Given the memory representation of a tree that stores the names of family members, construct the corresponding tree from the given data.

ROOT 3

7 AVAIL

	LEFT	NAMES	RIGHT
1	12	Pallav	–1
2			
3	9	Amar	13
4			
5			
6	19	Deepak	17
7			
8			
9	1	Janak	–1
10			
11	–1	Kuvam	–1
12	–1	Rudraksh	–1
13	6	Raj	20
14			
15	–1	Kunsh	–1
16			
17	–1	Tanush	–1
18			
19	–1	Ridhiman	–1
20	11	Sanjay	15



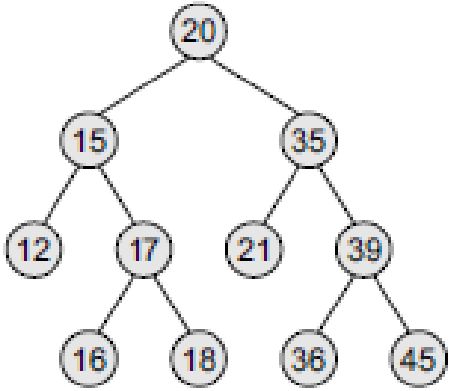
Sequential representation of binary trees

- Sequential representation of trees is done using single or one-dimensional arrays.
- Though it is the simplest technique for memory representation, it is inefficient as it requires a lot of memory space.
- A sequential binary tree follows the following rules:

Sequential representation of binary trees cont..

- A one-dimensional array, called TREE, is used to store the elements of tree.
- The root of the tree will be stored in the first location. That is, TREE[1] will store the data of the root element.
- The children of a node stored in location K will be stored in locations (2 × K) and (2 × K+1).
- The maximum size of the array TREE is given as (2h–1), where h is the height of the tree.
- An empty tree or sub-tree is specified using NULL. If TREE[1] = NULL, then the tree is empty.

The following figure shows a binary tree and its corresponding sequential representation. The tree has 11 nodes, and its height is 3.



1	20
2	15
3	35
4	12
5	17
6	21
7	39
8	
9	
10	16
11	18
12	
13	
14	36
15	45

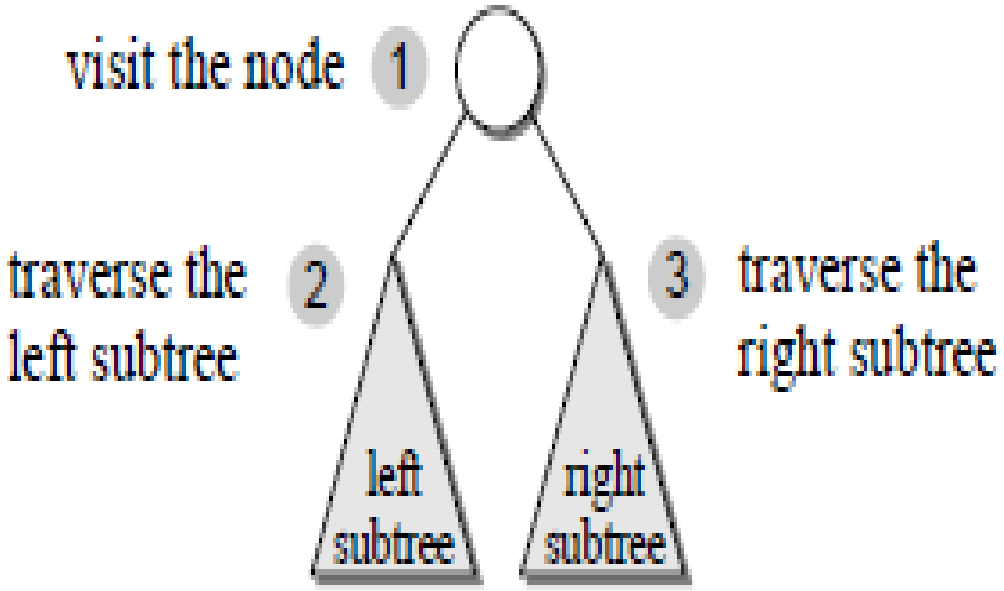
7.4 Traversals of Binary Trees

- A traversal of a tree is a systematic way of accessing or “visiting” all the nodes in the tree.
- There are three basic traversal schemes:
  - Pre-order traversal
  - In-order traversal
  - Post-order traversal

7.4.1 Pre-Order Traversal

- A pre-order traversal has three steps for a nonempty tree:
  - Process the root.
  - Process the nodes in the left subtree with a recursive call.
  - Process the nodes in the right subtree with a recursive call.

Pre-order traversal

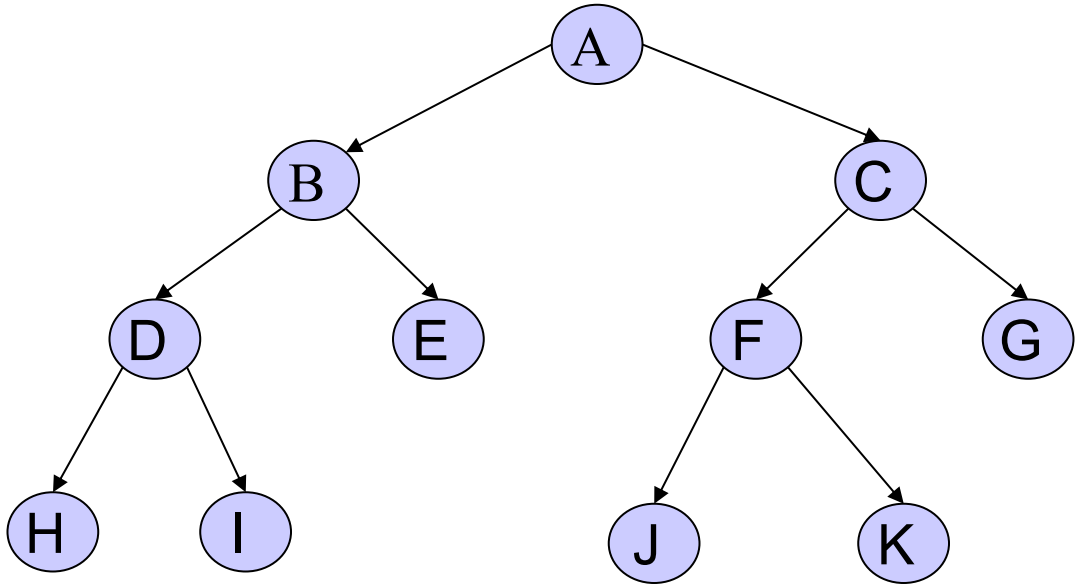


Pre-order traversal

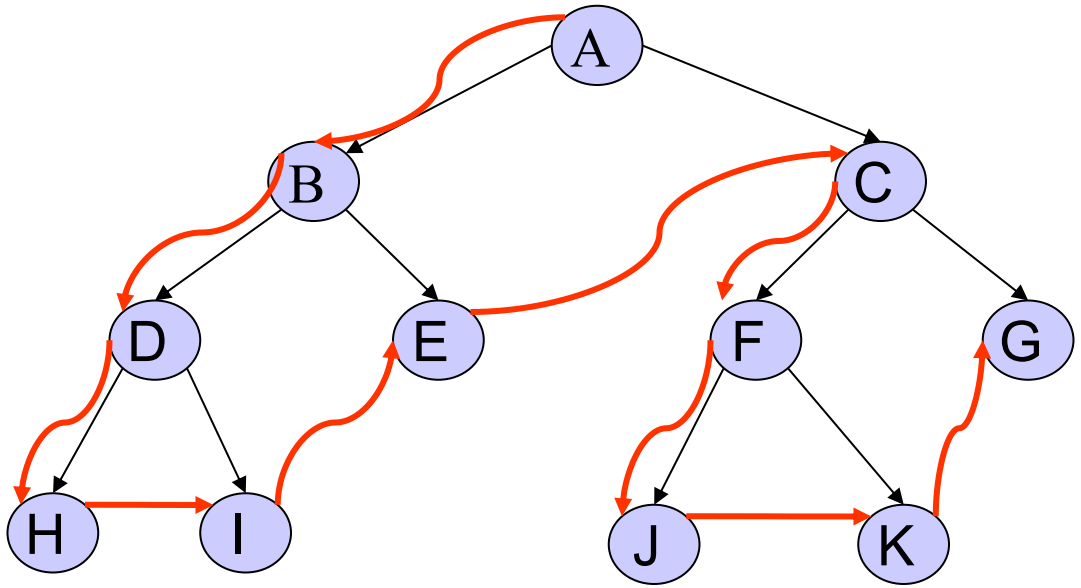
To traverse a non-empty binary tree in pre-order (also known as depth first order), we perform the following operations.

- Visit the root (or print the root)
- Traverse the left in pre-order (Recursive)
- Traverse the right tree in pre-order (Recursive)

7.4.1 Pre-Order Traversal

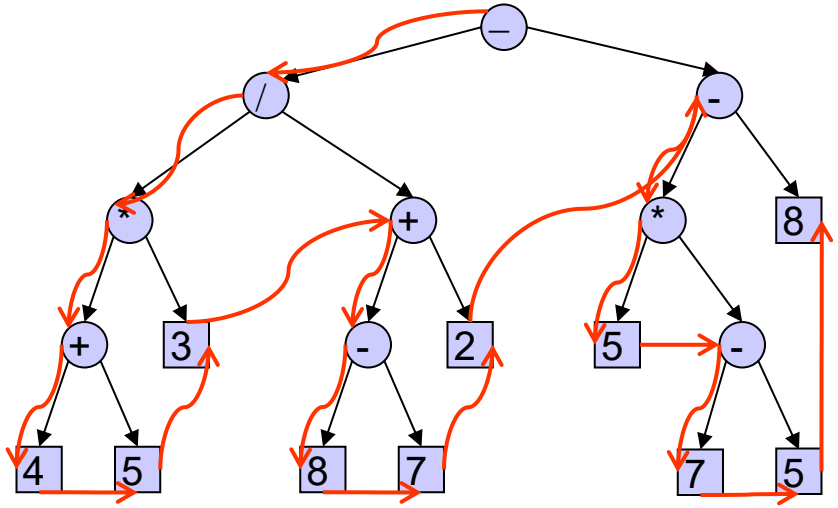


7.4.1 Pre-Order Traversal



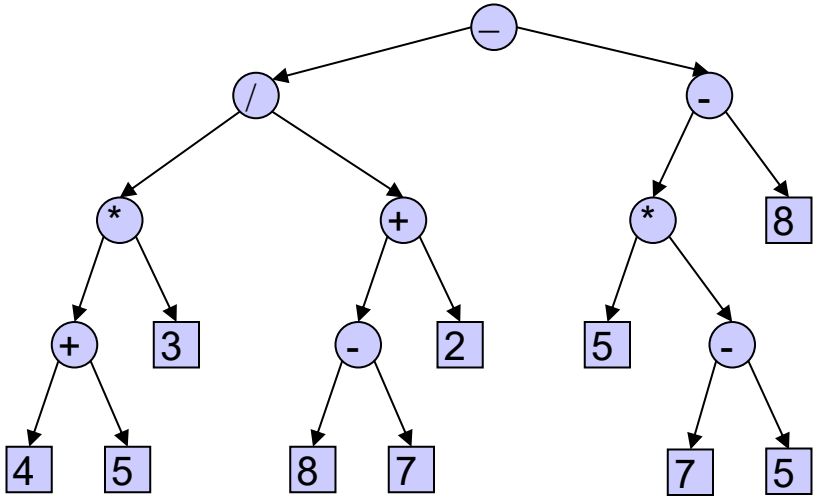
7.4.1 Pre-Order Traversal

- Using pre-order traversal of a binary tree to solve the expression evaluation problem.

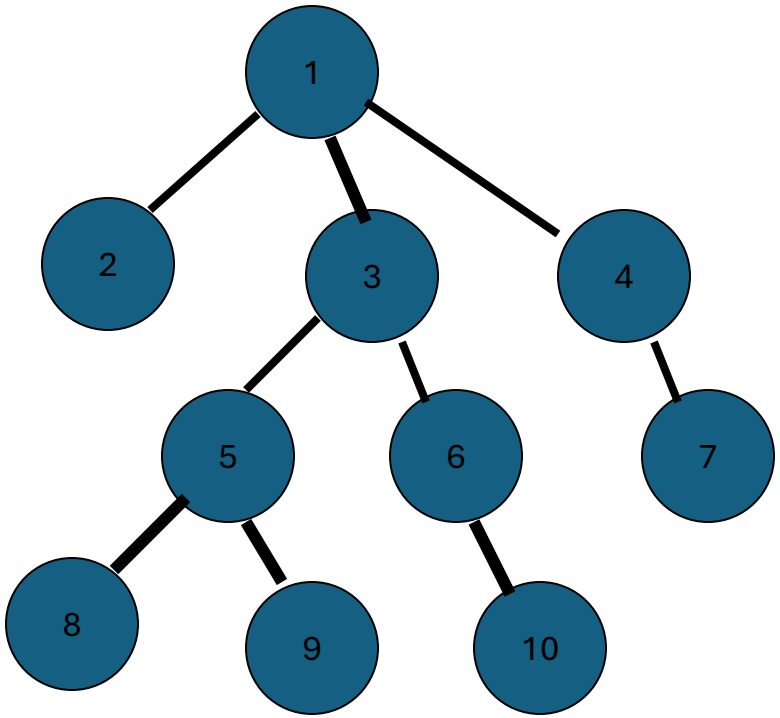


7.4.1 Pre-Order Traversal

- Using pre-order traversal of a binary tree to solve the expression evaluation problem.



Pre-order traversal



Pre-order list –1,2,3,5,8,9,6,10,4,7

Algorithm for pre-order traversal

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:            Write TREE -> DATA
Step 3:            PREORDER(TREE -> LEFT)
Step 4:            PREORDER(TREE -> RIGHT)
       [END OF LOOP]
Step 5: END
```

7.4.2 In-Order Traversal

- An in-order traversal has three steps for a nonempty tree:
  - Process the nodes in the left subtree with a recursive call.
  - Process the root.
  - Process the nodes in the right subtree with a recursive call.

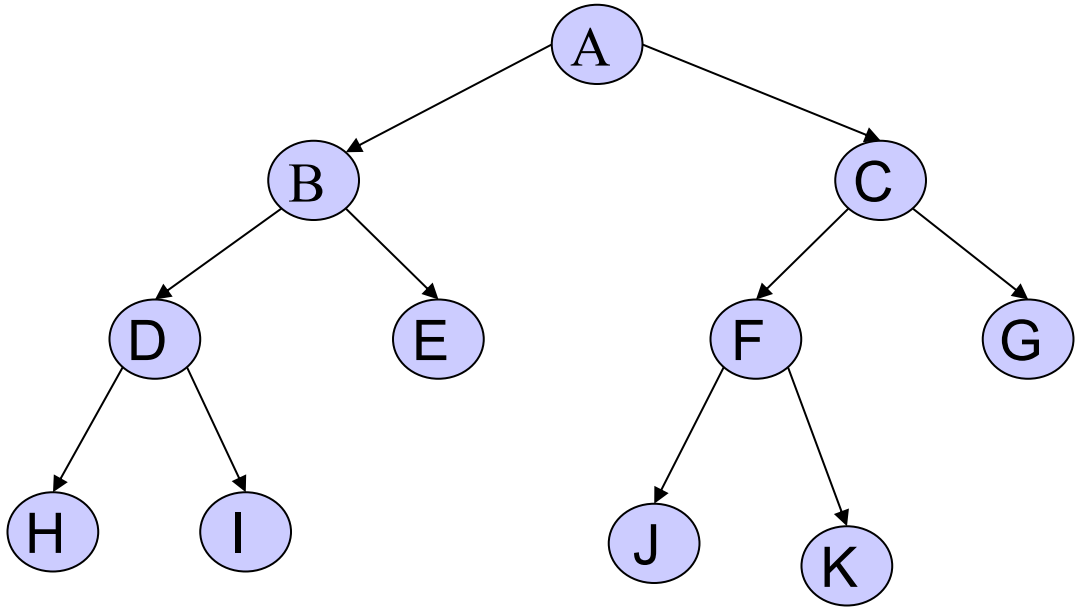
In-order traversal

The in-order listing of the nodes of T is the nodes of T_1 in in-order, followed by n,followed by the nodes T_1,T_2,....T_n, each group of nodes in in-order.

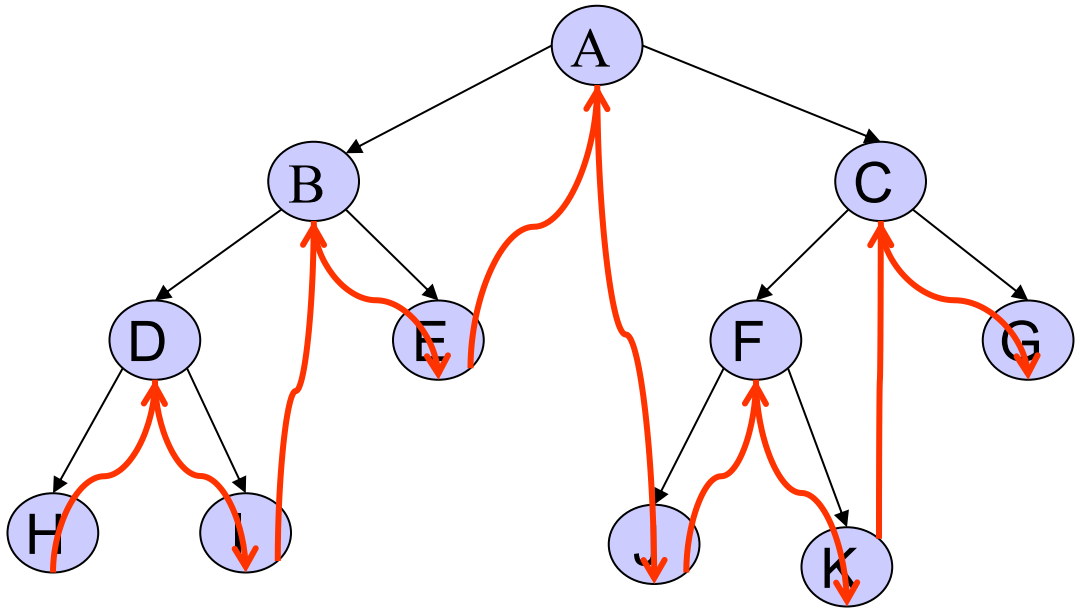
Algorithm :

- Traverse the left-subtree in in-order
- Visit the root
- Traverse the right-subtree in in-order.

7.4.2 In-Order Traversal

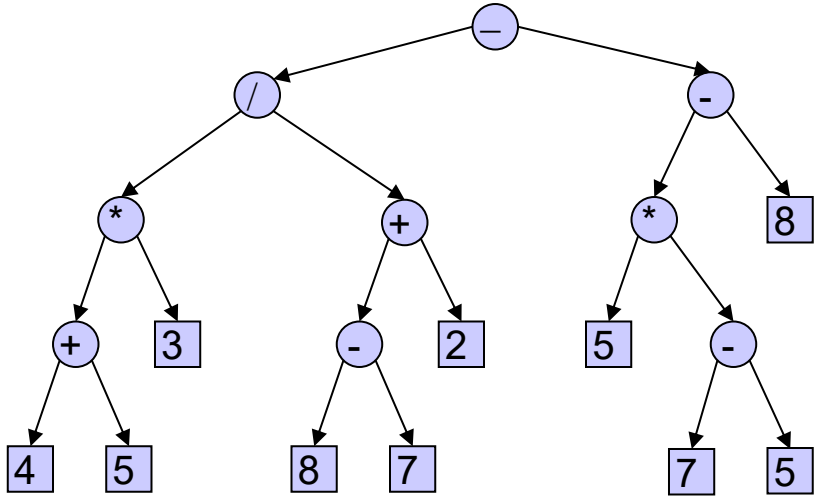


7.4.2 In-Order Traversal



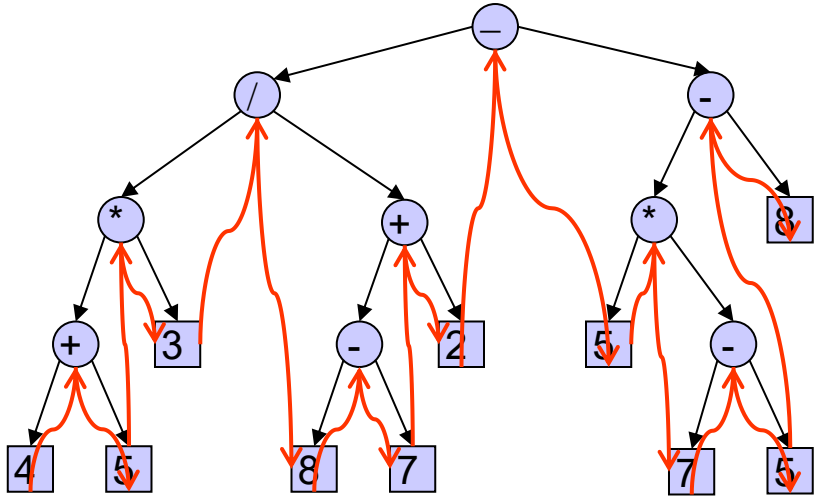
7.4.2 In-Order Traversal

- Using in-order traversal of a binary tree to solve the expression evaluation problem.

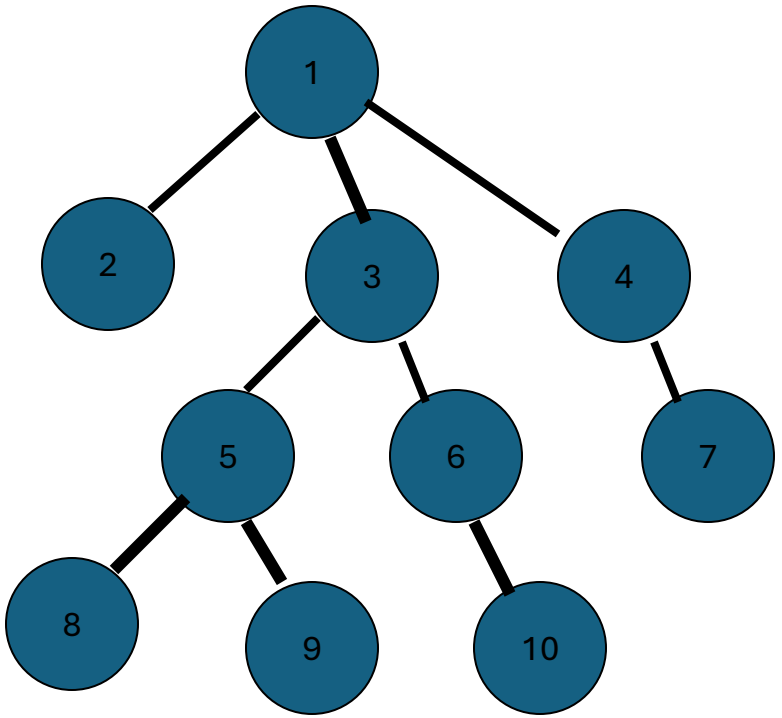


7.4.2 In-Order Traversal

- Using in-order traversal of a binary tree to solve the expression evaluation problem.



In-order traversal



In-order list –2,1,8,5,9,3,10,6,7,4

Algorithm for in-order traversal

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:                INORDER(TREE -> LEFT)
Step 3:                Write TREE -> DATA
Step 4:                INORDER(TREE -> RIGHT)
        [END OF LOOP]
Step 5: END
```

7.4.3 Post-Order Traversal

- A post-order traversal has three steps for a nonempty tree:
  - Process the nodes in the left subtree with a recursive call.
  - Process the nodes in the right subtree with a recursive call.
  - Process the root.

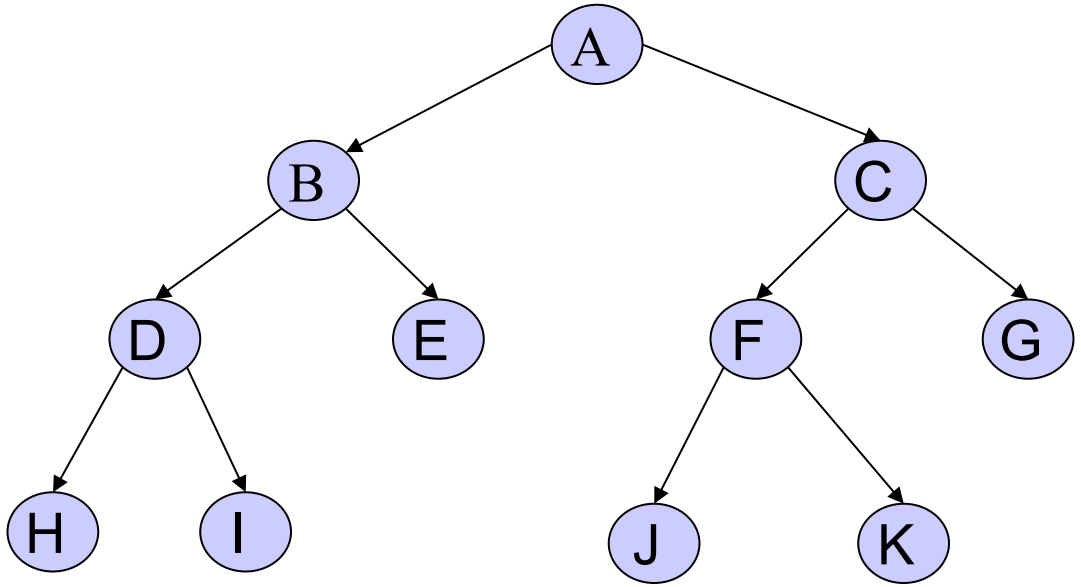
Post-order traversal

The post-order listing of the nodes of T is the nodes of T_1 in post-order, then the nodes of T_2 in post order and so-on up to T_k, all followed by n

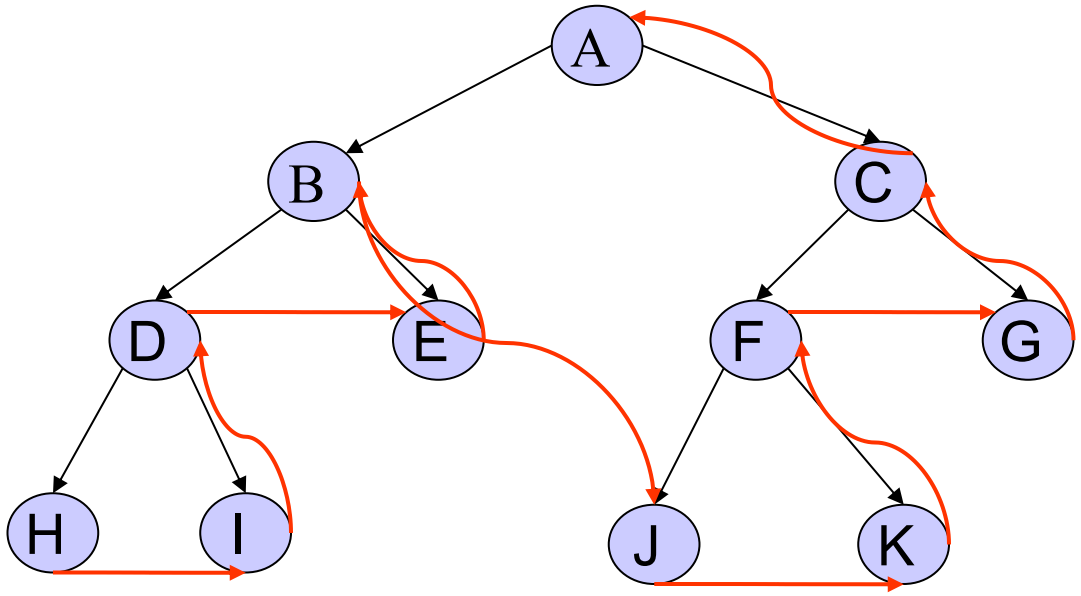
Algorithm

- Travers the left sub-tree in post-order
- Traverse the right sub-tree in post-order
- Visit the root

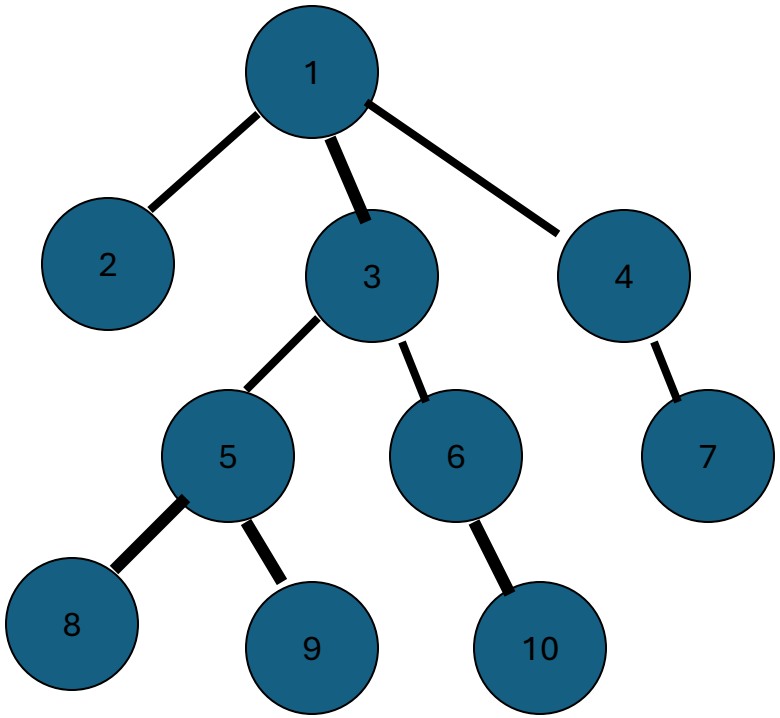
7.4.3 Post-Order Traversal



7.4.3 Post-Order Traversal



post-order traversal

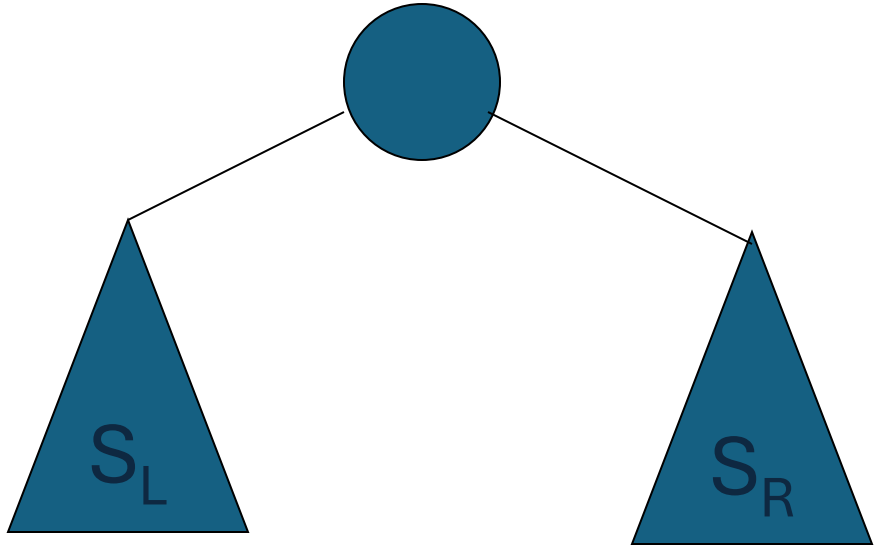


Pre-order list –2,8,9,5,10,6,3,7,4,1

Algorithm for post-order traversal

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:            POSTORDER(TREE -> LEFT)
Step 3:            POSTORDER(TREE -> RIGHT)
Step 4:            Write TREE -> DATA
       [END OF LOOP]
Step 5: END
```

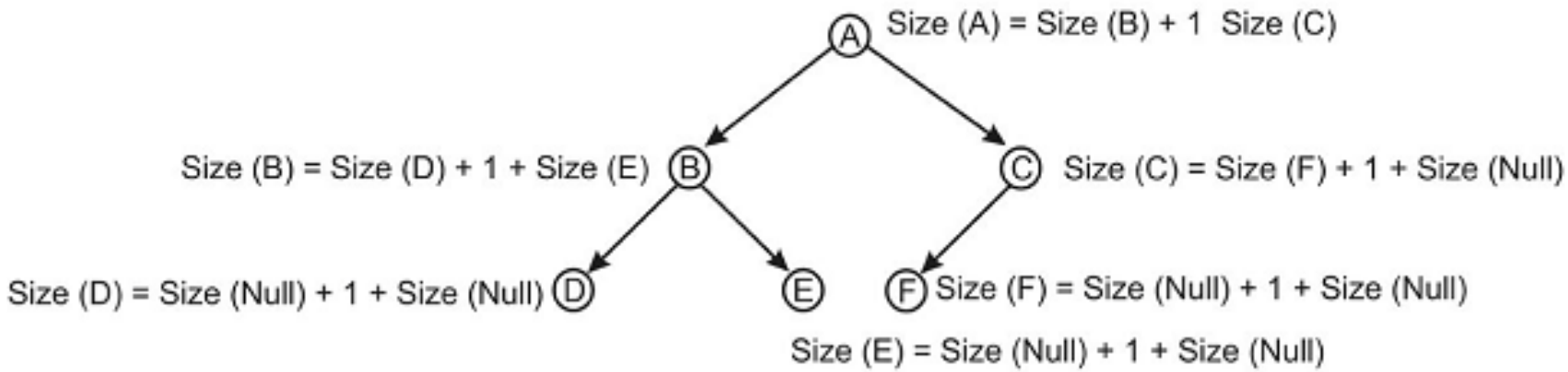
Recursive view used to calculate the size of a tree : $S_T = S_L + S_R + 1$



Recursive view used to calculate the size of a tree : $S_T=S_L+S_R+1$

- Below algorithm finds the size of a tree recursively.
- ie, size of a tree = size of a leftsubtree + 1 + size of a right subtree.
- For each Node, size() function calculates the size of the leftsubtree and rightsubtree and adds 1, to include the present node.

Example


A
Size (A) = Size (B) + 1 Size (C)
B
Size (B) = Size (D) + 1 + Size (E)
C
Size (C) = Size (F) + 1 + Size (Null)
D
Size (D) = Size (Null) + 1 + Size (Null)
E
Size (E) = Size (Null) + 1 + Size (Null)
F
Size (F) = Size (Null) + 1 + Size (Null)

Algorithm

- Same as Tree Traversal
- 1. If the tree is empty return 0
  2. else,
     a. return size of the tree ie, (size(Node->left)+ 1 + size(Node->right))
     In return statement we are calling the size of the left subtree recursively ie, size(Node->left)
     In return statement we are calling the size of right subtree recursively ie, size(Node->right)

Pseudo Code Algorithm to calculate the size of the binary tree

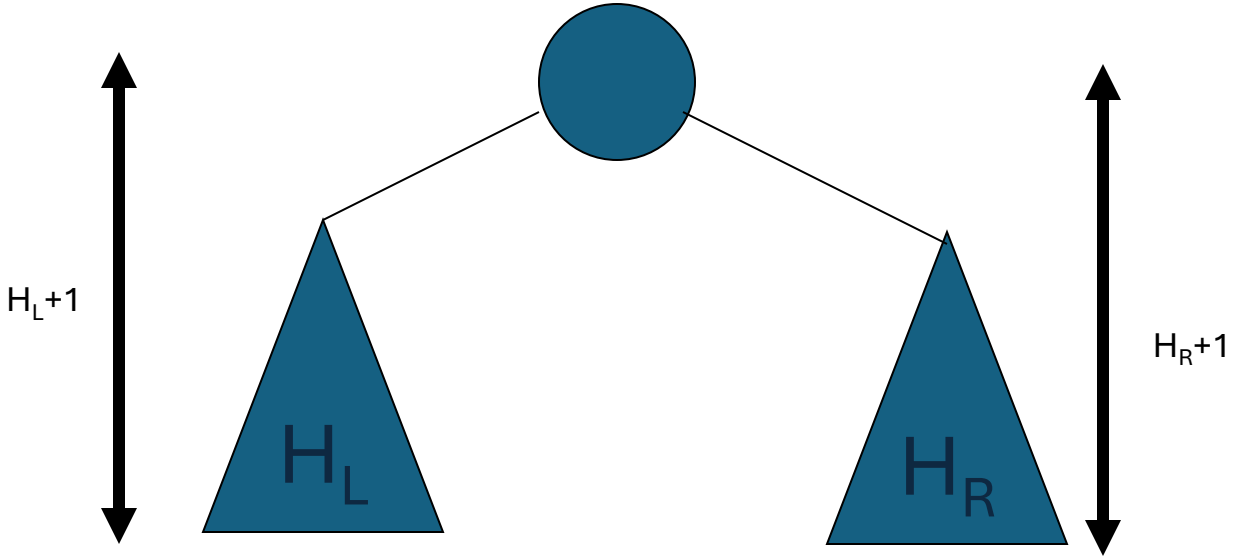
```
//* Return the size of the binary tree rooted at t.
size (BinaryNode t)
if  (t = = null)
return 0
else
return 1+size(t->left)+ size(t->right)
endif
```

C++ Program

```
#include<iostream>
using namespace std;

struct Node  //Defining a Node
{
        char data;
        Node* left;
        Node* right;
};

Node* newNode(char data){  //Creating a Node with left and right pointers pointing to N
        Node* node = new Node();
        node->data = data;
        node->left = NULL;
        node->right = NULL;
        return node;
}
```

```
}
int size(Node* node) //Function which returns the size of the tree
{
        if(node!=NULL) //For each node calculate the size of left subtree and right sub
        {
                return(size(node->left)+1+size(node->right)); //Size of tree= Size of l
        }
        else
        {
                //cout<<"hi";
                return 0; //If there is no node return 0 as the size
        }

}
int main()
{
        Node* root = newNode('A');  //Assigning the Node values
        root->left = newNode('B');
        root->right = newNode('C');
        root->left->left = newNode('D');
        root->left->right = newNode('E');
        root->right->left = newNode('F');
        cout<<"size of the tree is "<<size(root); //Printing the size of the tree
        return 0;

}
```

Recursive view used to calculate the height of a tree : $H_T = \max(H_L + 1, H_R + 1)$



Routine to compute the height of a node (figure 1.2)

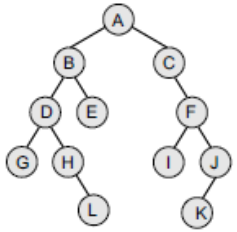
- //* Return the height of the binary tree rooted at t.
- height (BinaryNode t)
- If (t = = null)
- return 0;
- else
- return 1+max(height(t->left),height(t->right);
- }

Level-order Traversal

- In level-order traversal, all the nodes at a level are accessed before going to the next level.
- This algorithm is also called as the *breadth-first traversal(BFT/BFS) algorithm*.
- Consider the trees given in the following figures and note the level order of these trees.

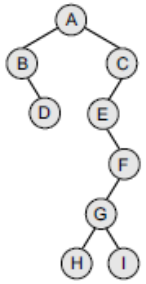
(a)

TRAVERSAL ORDER:
A, B, and C



(b)

TRAVERSAL ORDER:
A, B, C, D, E, F, G, H, I, J, L, and K



(c)

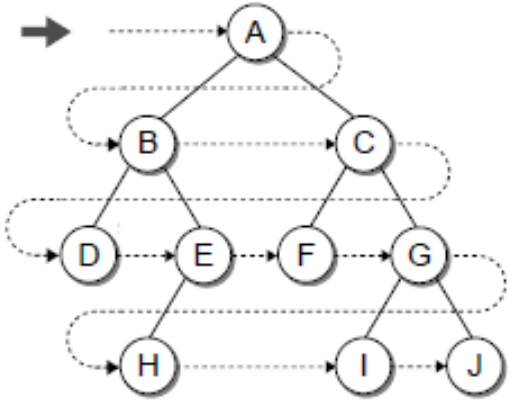
TRAVERSAL ORDER:
A, B, C, D, E, F, G, H, and I

Breadth-First Traversal

- The pre-order, in-order, and post-order traversals are all examples of a depth-first traversal.
- That is, the nodes are traversed deeper in the tree before returning to higher-level nodes.
- Another type of traversal that can be performed on a binary tree is the **breadth-first traversal**.

Breadth-First Traversal

- In the breadth-first traversal, the nodes are visited level by level, from left to right.



The logical ordering of the nodes with a breadth-first traversal

Breadth-First Traversal

- Recursion cannot be used to implement a breadth-first traversal since the recursive calls must follow the links that lead deeper into the tree.
- The best way to save a node's children for later access is to use a queue.

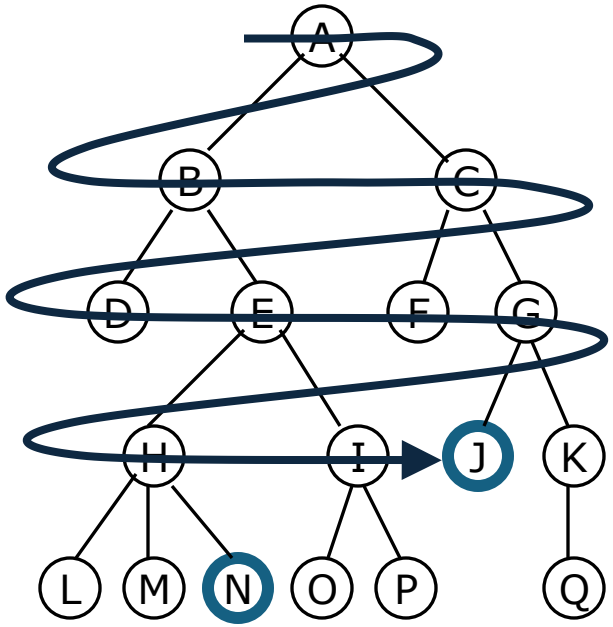
Implementation of Breadth-first-traversal

- In the above example tree, we would visit node A followed by nodes B and C, which is the correct ordering.
- The best way to save a node's children for later access is to use a queue.
- We can then use an iterative loop to move across the tree in the correct node order to produce a breadth-first traversal.

Breadth-First Traversal

- The above Listing uses a queue to implement the breadth-first traversal.
- The process starts by saving the root node and in turn priming the iterative loop.
- During each iteration, we remove a node from the queue, visit it, and then add its children to the queue.
- The loop terminates after all nodes have been visited.

Breadth-first searching

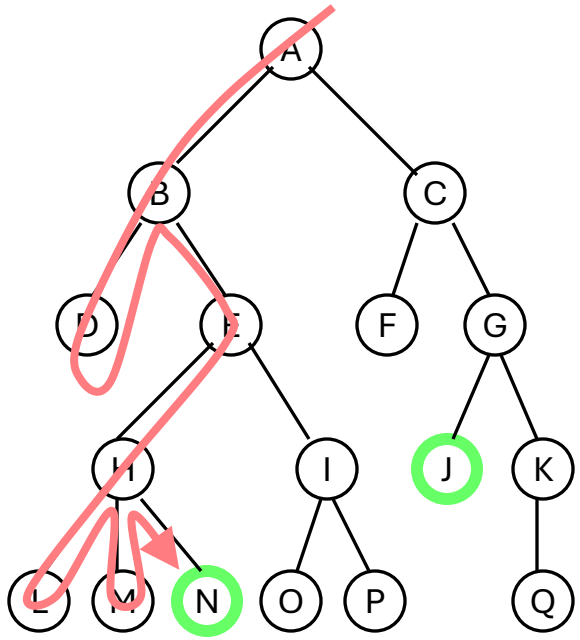


- A breadth-first search (BFS) explores nodes nearest the root before exploring nodes further away
- For example, after searching A, then B, then C, the search proceeds with D, E, F, G
- Node are explored in the order A B C D E F G H I J K L M N O P Q
- J will be found before N

Pseudo-Code for Breadth-First Traversal

```
breadth-first-traversal
    put root node onto a queue
    while the queue is not empty
        dequeue the next node
        visit the node                    e.g., print value
        enqueue the left child node
        enqueue the right child node
```

Depth-first searching



- A depth-first search (DFS) explores a path all the way to a leaf before backtracking and exploring another path
- For example, after searching A, then B, then D, the search backtracks and tries another path from B
- Node are explored in the order A B D E H L M N I O P C F G J K Q
- N will be found before J

Depth First Traversals

- Inorder Traversal (Left-Root-Right)
- Preorder Traversal (Root-Left-Right)
- Postorder Traversal (Left-Right-Root)

Expression trees

- One use of the binary tree is in the expression tree, which is central data structure in compiler design.
- The structure of the expression tree is based on the order in which the operators are evaluated. The operator in each internal node is evaluated after both its left and right subtrees have been evaluated.

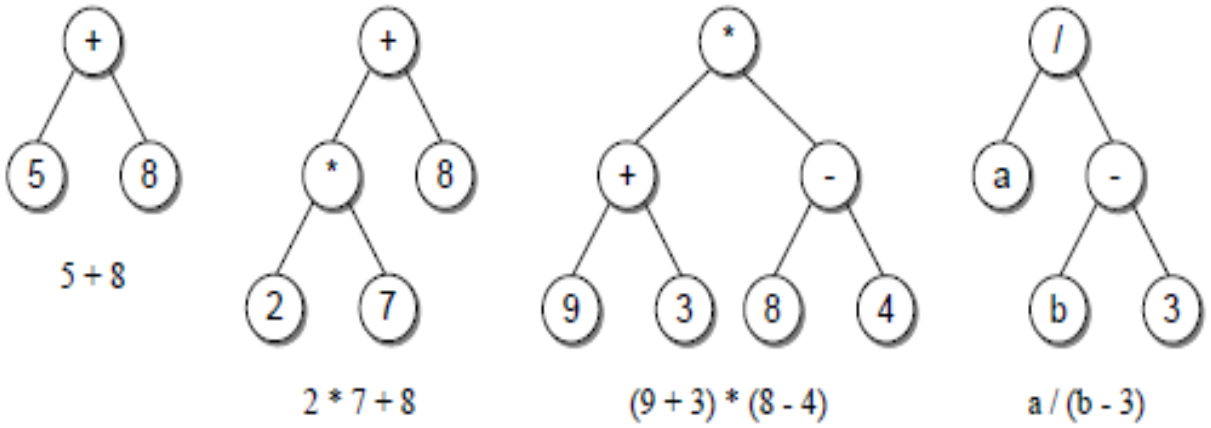
Expression Trees

- Arithmetic expressions such as (9+3)*(8-4) can be represented using an expression tree.
- An expression tree is a binary tree in which the operators are stored in the interior nodes and the operands (the variables or constant values) are stored in the leaves.

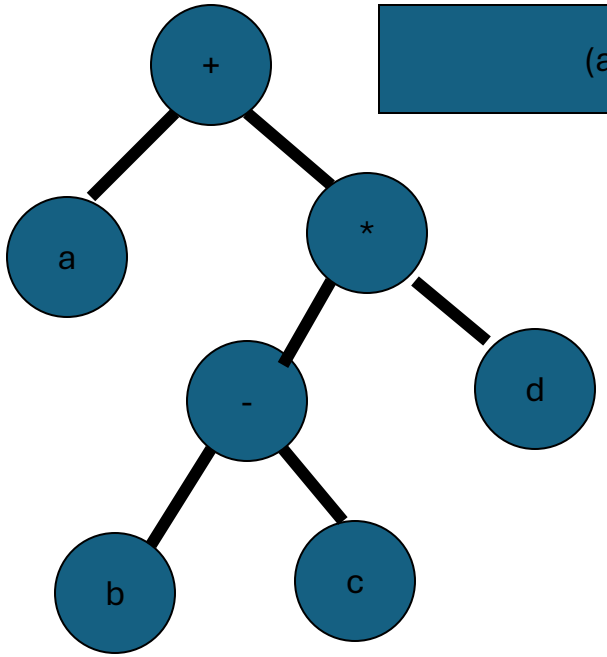
Expression Trees

- Once constructed, an expression tree can be used to evaluate the expression or for converting an infix expression to either pre-fix or postfix notation.

Examples :Expression tree



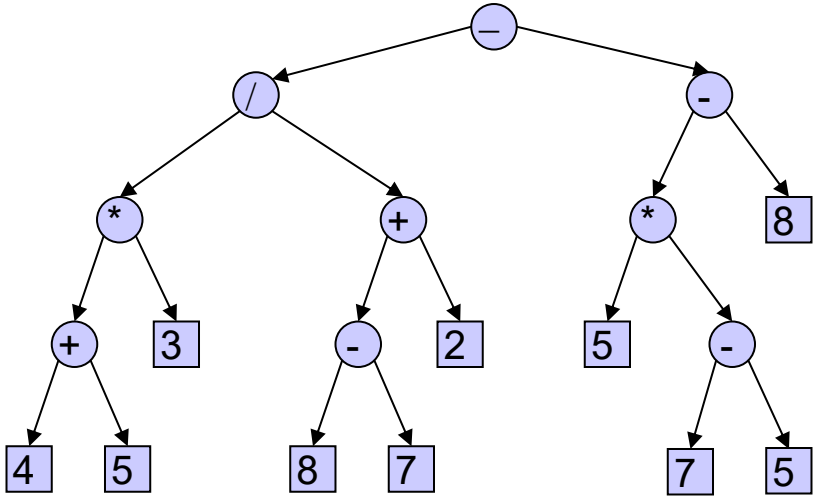
Example :Expression tree



(a+((b-c)*d))

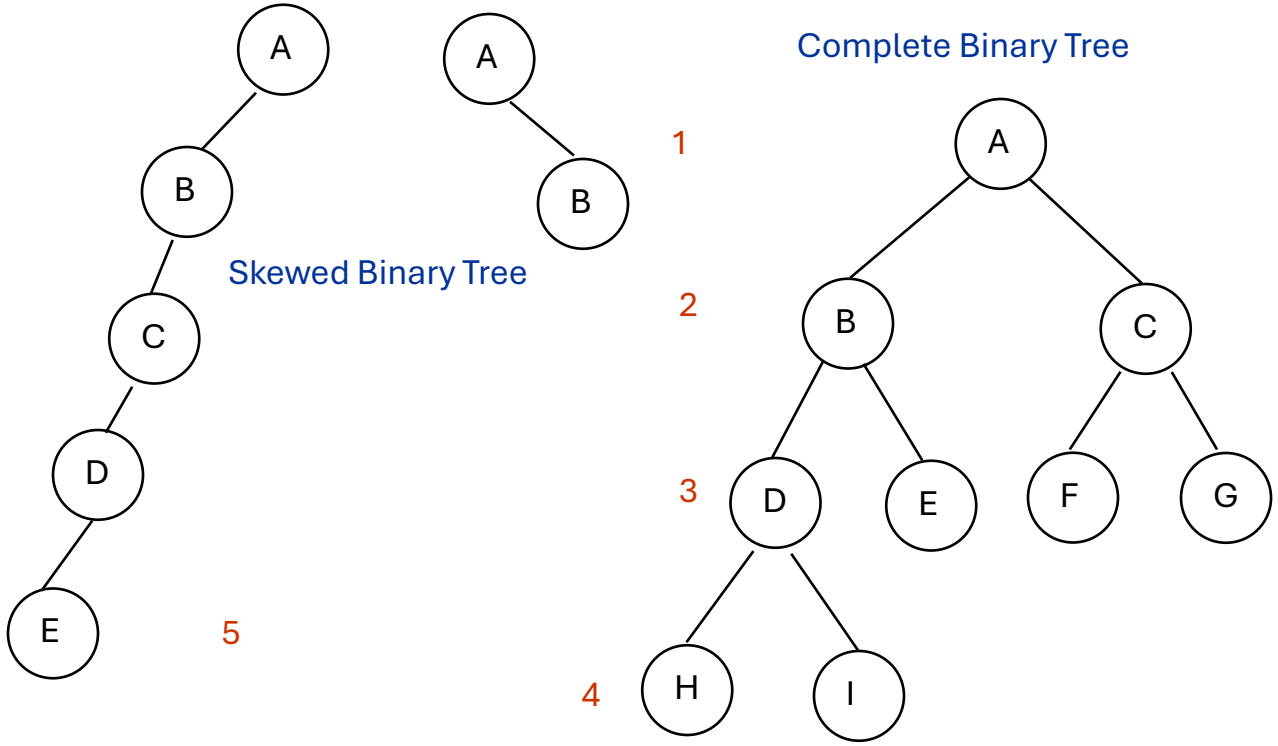
The leaves of an expression tree are operands, such as constant, variable names. The other nodes contain operators.

Given a expression (((4+5)*3)/((8-7)+2))-((5*(7-5))-8), Its expression tree is as follows:

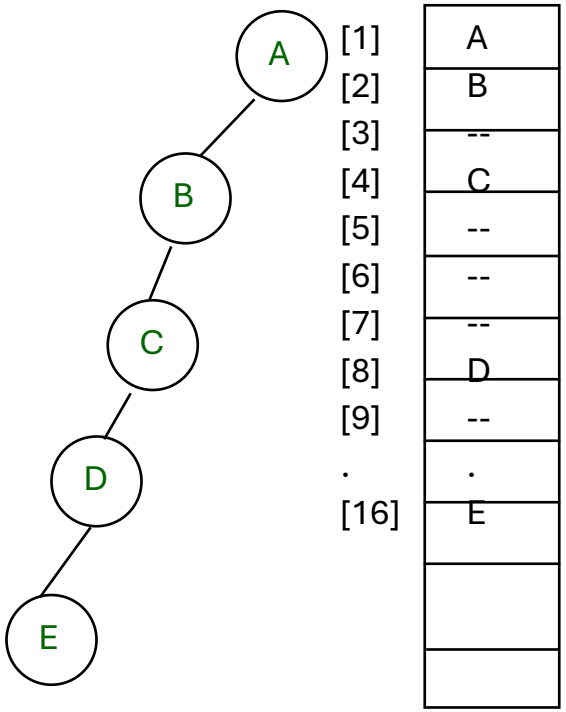


Implementation

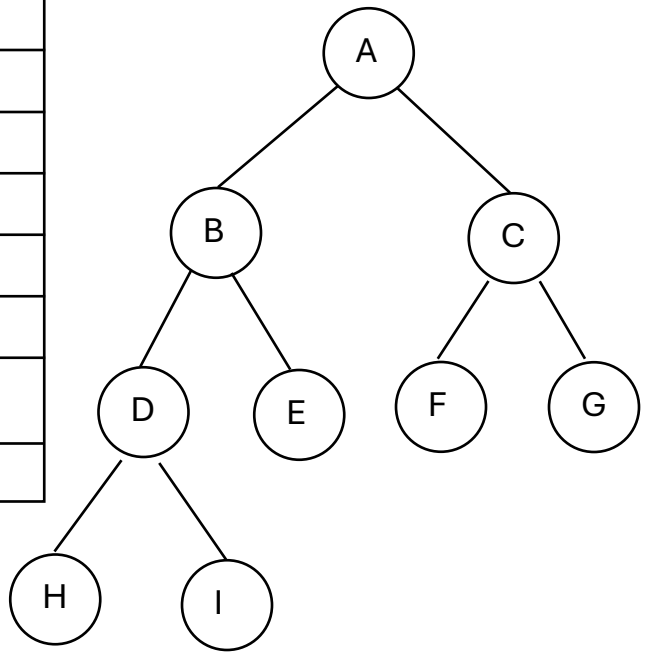
Samples of Trees

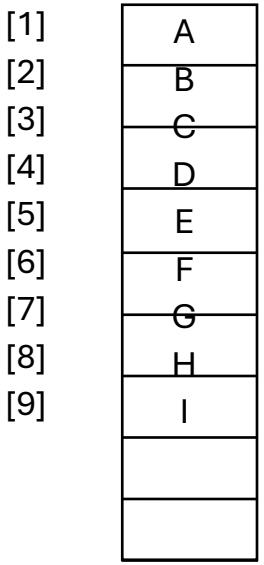


Sequential Representation



(1) waste space
(2) insertion/deletion problem





Expression Tree Construction

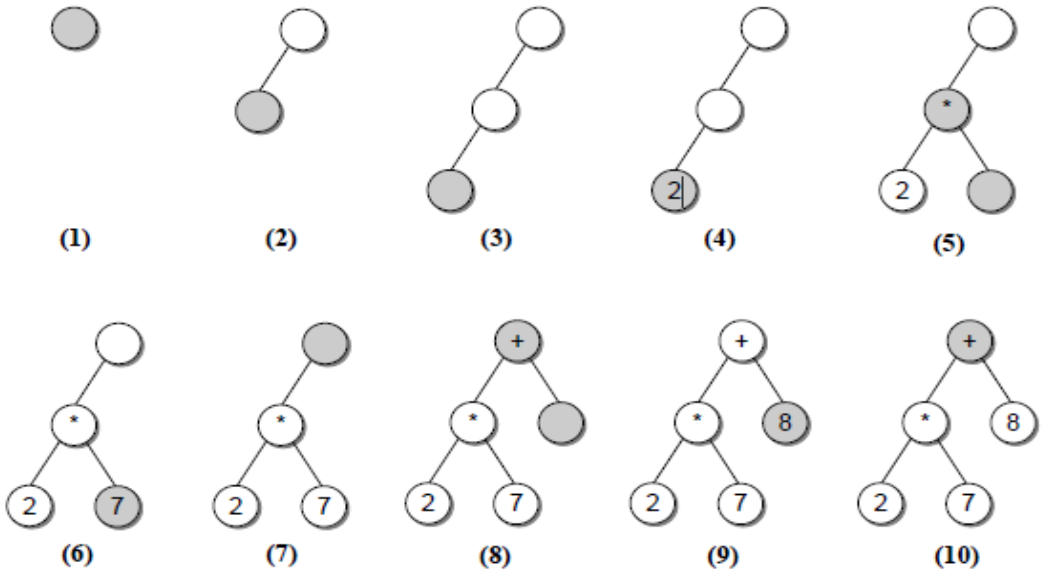
- Consider the following Figure, which illustrates the steps required to build the tree for the expression ((2*7)+8).
- The steps illustrated in the Figure are described below:

Steps for building an expression tree for ((2* 7) + 8).

1. Create an empty root node and mark it as the current node.
2. Read the left parenthesis: add a new node as the left child and descend down to the new node.
3. Read the next left parenthesis: add a new node as the left child and descend down to the new node.
4. Read the operand 2: set the value of the current node to the operand and move up to the parent of the current node.
5. Read the operator $*$: set the value of the current node to the operator and create a new node linked as the right child. Then descend down to the new node.
6. Read the operand 7: set the value of the current node to the operand and move up to the parent of the current node.
7. Read the right parenthesis: move up to the parent of the current node.
8. Read the operator $+$: set the value of the current node to the operator and create a new node linked as the right child; descend down to the new node.

Steps for building an expression tree for ((2* 7) + 8).

9. Read the operand 8: set the value of the current node to the operand and move up to the parent of the current node.

10. Read the right parenthesis: move up to the parent of the current node. Since this is the last token, we are finished and the expression tree is complete.

Steps for building an expression tree for ((2 * 7) + 8).